Deep Learning

https://chatgpt.com/share/69935793-8f70-800c-864c-dc2e1dd63b71

ADL

🧠 PART 1 — Deep Learning Fundamentals (Start from ZERO)

1. What is Learning in Machine Learning?

Goal:
We want computer to learn a function

[
y = f(x)
]

Where

Symbol	Meaning
(x)	Input data (image, text, number etc.)
(y)	Correct output (label)
(f)	Model (brain of AI)

Example
x = image → y = cat

So learning = finding best function (f)

2. ML vs DL (Core Idea)

Machine Learning (Old way)

Human extracts features manually

Example features of face:

- nose length
- eye distance
- color histogram

Then classifier learns

Deep Learning (Modern way)

Computer learns features itself

[
Image → Layers → Features → Decision
]

That is why it needs:

- large data
- GPU
- many parameters

---

3. What is a Neural Network?

Artificial neuron imitates brain neuron

[
z = w_1x_1 + w_2x_2 + w_3x_3 + b
]

[
a = \sigma(z)
]

Meaning of symbols

Symbol	Name	Meaning
(x_i)	input	feature value
(w_i)	weight	importance of feature
(b)	bias	adjustment constant
(z)	pre-activation	linear combination
(a)	activation	neuron output
(\sigma)	activation function	non-linearity

Why activation function needed?

Without activation:

[
f(x) = wx + b
]

Always straight line → cannot learn complex patterns

So we use nonlinear functions.

Common activation functions

Sigmoid

[
\sigma(z)=\frac{1}{1+e^{-z}}
]
Range: (0,1)
Used for probability

ReLU (Most Important)

[
ReLU(z)=\max(0,z)
]

Best because:

- fast
- avoids vanishing gradient

🧠 PART 2 — Training the Network

Loss Function (Error)

Measures wrongness of prediction

For regression

[
MSE = \frac{1}{n}\sum (y - \hat{y})^2
]

Symbol	Meaning
(y)	true value

Symbol	Meaning
(\hat{y})	predicted value
(n)	number of samples

Goal:

[
\min Loss
]

Gradient Descent (Learning Rule)

We update weights:

[
w = w - \eta \frac{\partial L}{\partial w}
]

Symbol	Meaning
(L)	loss
(\eta)	learning rate
(\partial L/\partial w)	gradient

Types

Type	Idea
Batch GD	whole dataset
SGD	one sample
Mini-batch	small chunk (best)

🧠 PART 3 — Backpropagation (How network learns)

Chain rule:

[
\frac{\partial L}{\partial w}

\frac{\partial L}{\partial a}
\frac{\partial a}{\partial z}
\frac{\partial z}{\partial w}
]

This sends error from output → input

Called **back propagation**

Vanishing Gradient Problem

Sigmoid derivative:

[
\sigma'(z)=\sigma(z)(1-\sigma(z)) \le 0.25
]

When many layers multiply small numbers:

[
0.25 × 0.25 × 0.25 × ... → 0
]

Network stops learning ❌

Solution → ReLU ✓

🧠 PART 4 — Convolutional Neural Network (CNN)

Used for images

Why not normal ANN for images?

Image 200×200 = 40,000 inputs
Fully connected → millions of weights

Impossible to train

Convolution Operation

[
Feature = Image * Kernel
]

Kernel slides across image

Symbols

Symbol	Meaning
(I)	image
(K)	filter/kernel
(F)	feature map
(S)	stride
(P)	padding

Output size:

[
\frac{N - F + 2P}{S} + 1
]

Pooling Layer

Reduces size

Max pooling:

[
\max(window)
]

Purpose:

- reduce computation
- keep important info

CNN Pipeline

[
Input → Conv → ReLU → Pool → FC → Softmax
]

🧠 PART 5 — Famous Architectures

AlexNet

- First deep CNN success

VGG

- only 3×3 filters
- very deep

GoogleNet (Inception)

- multiple filters parallel

ResNet

Key idea:

[
Output = F(x) + x
]

Skip connection solves degradation

🧠 PART 6 — Autoencoder (Now your Sparse AE topic)

What is Autoencoder?

Neural network that copies input to output

[
x → Encoder → Latent → Decoder → \hat{x}
]

Goal:

[
\hat{x} \approx x
]

Used for:

- compression
- feature learning
- denoising

Architecture

Encoder:
[
h = \sigma(W_e x + b_e)
]

Decoder:
[
\hat{x} = W_d h + b_d
]

Symbols

Symbol	Meaning
(x)	input vector
(h)	latent representation
(W_e)	encoder weights
(b_e)	encoder bias
(W_d)	decoder weights
(b_d)	decoder bias
(\hat{x})	reconstruction

Reconstruction Loss

[
L_{rec} = \frac{1}{n}\sum (x-\hat{x})^2
]

🧠 PART 7 — Sparse Autoencoder (Your pages)

Normal autoencoder may copy everything
We want neuron activate rarely → learn meaningful features

So we enforce sparsity

Step 1 — Average activation

$$
\hat{\rho_j} = \frac{1}{m}\sum_{i=1}^{m} a_j(x^{(i)})
$$

Symbol	Meaning
j	neuron index
m	training samples
a_j	activation of neuron j
$\hat{\rho_j}$	average activation

We want:

$$
\hat{\rho_j} \approx \rho
$$

Example target sparsity:

$$
\rho = 0.05
$$

Neuron active only 5% times

Step 2 — KL Divergence penalty

$$KL(\rho||\hat{\rho_j})$$

$$
\rho \log\frac{\rho}{\hat{\rho_j}}
+
(1-\rho)\log\frac{1-\rho}{1-\hat{\rho_j}}
$$

Measures distance between desired and actual activation

Step 3 — Total Loss

$$
L = Reconstruction\ Loss + \beta \sum KL(\rho||\hat{\rho_j})
$$

Symbol	Meaning
(\beta)	sparsity weight
KL	sparsity penalty

Intuition (Practical Understanding)

Normal autoencoder → remembers image
Sparse autoencoder → learns features like:

- edges
- strokes
- patterns

Because few neurons activate at once

🧠 PRACTICAL VIEW (How question solved in exam)

Steps:

1. Forward pass → compute hidden layer
   [
   h = \sigma(W_e x + b_e)
   ]
2. Reconstruct
   [
   \hat{x} = W_d h + b_d
   ]
3. Compute reconstruction loss (MSE)
4. Compute average activation
5. Compute KL divergence
6. Total loss

[
L = MSE + \beta KL
]

7. Count parameters
   Encoder params = inputs×hidden + hidden bias
   Decoder params = hidden×output + output bias

⭐ TOP 20 IMPORTANT QUESTIONS

🔹 SET-1 (Questions 1-5)

🔶 QUESTION 1

Derive the output of a single artificial neuron using sigmoid activation for given inputs and weights.

STEP-1: THEORY EXPLANATION (FROM ZERO)

Intuition

Think of a neuron like a **decision maker**.

Example:
You decide whether to study based on:

- Time available
- Exam near
- Mood

Each factor has importance → that is weight.

So neuron works as:

```
inputs → multiply importance → add adjustment → make decision
```

Basic Working of Artificial Neuron

```
x1 ----(w1)---
               \
x2 ----(w2)---- > [ SUM ] → [ Activation ] → Output
```

```
              /
x3 ----(w3)---
        + bias
```

It does 4 steps:

1. Multiply input with weight
2. Add all values
3. Add bias (threshold shifting)
4. Apply activation function

Why Activation Needed?

Without activation:

Model only draws straight line
But real world needs curves

So activation adds intelligence.

Where used?

- CNN fully connected layer
- Autoencoder hidden layer
- Backpropagation learning

STEP-2: MATHEMATICAL FORMULAS WITH SYMBOL NAMES

Weighted sum

[
z = w_1x_1 + w_2x_2 + w_3x_3 + b
]

Symbol	Meaning
(x_i)	input feature
(w_i)	weight (importance)
(b)	bias (shift value)
(z)	pre-activation value

Sigmoid Activation

$$
a = \sigma(z) = \frac{1}{1+e^{-z}}
$$

Symbol	Meaning
(\sigma) (sigma)	sigmoid function
(e)	Euler constant (≈2.718)
(a)	final neuron output

STEP-3: NUMERICAL / CALCULATION

Given:

$$
x = [1,;0,;1]
$$

$$
w = [0.5,;0.3,;0.8]
$$

$$
b = 0.2
$$

Step-1: Multiply input and weight

$$
w_1x_1 = 0.5×1 = 0.5
$$

$$
w_2x_2 = 0.3×0 = 0
$$

$$
w_3x_3 = 0.8×1 = 0.8
$$

Step-2: Add all values

$$
z = 0.5 + 0 + 0.8 + 0.2
$$

$$
z = 1.5
$$

Step-3: Apply sigmoid

$$
a = \frac{1}{1+e^{-1.5}}
$$

$$
e^{-1.5} = 0.223
$$

$$
a = \frac{1}{1+0.223}
$$

$$
a = \frac{1}{1.223}
$$

$$
a = 0.817
$$

STEP-4: FINAL ANSWER

👉 **FINAL ANSWER: 0.817**

🔶 QUESTION 2

Explain ReLU activation and compute output for given vector.

STEP-1: THEORY EXPLANATION

Problem with sigmoid

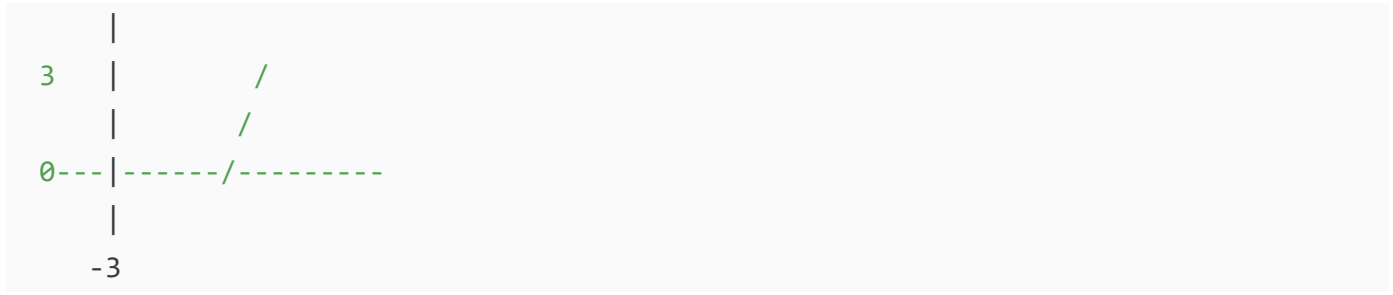
In deep networks gradients become very small → learning stops.

Solution → ReLU

ReLU acts like a switch:

```
Negative → OFF
Positive → PASS
```

Graph:



Where used?

- CNN convolution layers
- Deep networks
- ResNet

STEP-2: FORMULA & SYMBOLS

[
ReLU(z)=\max(0,z)
]

Symbol	Meaning
(z)	neuron input
max	choose larger value

STEP-3: NUMERICAL

Given vector:

[
[-2,;0.5,;3]
]

[
ReLU(-2)=0
]

[
ReLU(0.5)=0.5

]

[
ReLU(3)=3
]

STEP-4: FINAL ANSWER

👉 **FINAL ANSWER: [0,;0.5,;3]**

🔶 QUESTION 3

Compute Mean Squared Error (MSE) loss for predicted outputs.

STEP-1: THEORY EXPLANATION

Loss = How wrong the prediction is

Example:
Teacher marks paper → difference between correct & student answer

MSE punishes big mistakes more

Used in:

- Autoencoder reconstruction
- Regression problems

STEP-2: FORMULAS

[
MSE = \frac{1}{n}\sum (y_i-\hat{y_i})^2
]

Symbol	Meaning
(y_i)	true value
(\hat{y_i})	predicted value
(n)	number of samples
(\Sigma)	summation

STEP-3: NUMERICAL

True:

$$
[
[1,0,1,1]
]
$$

Predicted:

$$
[
[0.8,0.2,0.7,0.6]
]
$$

Calculate one-by-one

$$
[
(1-0.8)^2 = 0.2^2 = 0.04
]
$$

$$
[
(0-0.2)^2 = (-0.2)^2 = 0.04
]
$$

$$
[
(1-0.7)^2 = 0.3^2 = 0.09
]
$$

$$
[
(1-0.6)^2 = 0.4^2 = 0.16
]
$$

Sum

$$
[
0.04+0.04+0.09+0.16=0.33
]
$$

$$
[
MSE=\frac{0.33}{4}=0.0825
]
$$

STEP-4: FINAL ANSWER

👉 **FINAL ANSWER: 0.0825**

🔶 QUESTION 4

Update weight using Gradient Descent rule.

STEP-1: THEORY EXPLANATION

Goal: reduce loss

Imagine going downhill to reach valley (minimum error)

```
  /\
 /  \   ← loss
/    \__
```

Each step:

[
New\ weight = Old\ weight - step\ size × slope
]

Used in:

- Backpropagation
- CNN training
- Autoencoder training

STEP-2: FORMULA

[
w_{new}=w_{old}-\eta\frac{\partial L}{\partial w}
]

Symbol	Meaning
(\eta) (eta)	learning rate
(\partial)	partial derivative
(L)	loss
(w)	weight

STEP-3: NUMERICAL

Given:

$$
w=0.5,;\eta=0.1,;\frac{\partial L}{\partial w}=0.8
$$

$$
w_{new}=0.5-0.1\times0.8
$$

$$
w_{new}=0.5-0.08
$$

$$
w_{new}=0.42
$$

STEP-4: FINAL ANSWER

👉 **FINAL ANSWER: 0.42**

🔶 QUESTION 5

Explain Vanishing Gradient Problem with numerical demonstration.

STEP-1: THEORY EXPLANATION

During backpropagation gradients multiply across layers:

$$
gradient = small \times small \times small \times small
$$

Sigmoid derivative max = 0.25

So deep network:

$$
0.25^{10} \approx 0
$$

Weights stop updating → network forgets learning

Solution:

- ReLU
- ResNet skip connection

STEP-2: FORMULA

$$
\sigma'(z)=\sigma(z)(1-\sigma(z))
$$

Symbol	Meaning
σ	sigmoid
$'$	derivative
z	neuron input

STEP-3: NUMERICAL

$$
0.25^5
$$

$$
=0.25\times0.25\times0.25\times0.25\times0.25
$$

$$
=0.000976
$$

Very small → gradient vanished

STEP-4: FINAL ANSWER

👉 **FINAL ANSWER: Gradients become almost zero in deep sigmoid networks, stopping learning**

🔶 SET-2 (Q6–Q10) — BACKPROPAGATION & CNN

🔶 QUESTION 6

Explain Backpropagation algorithm and compute weight update for single neuron.

STEP-1: THEORY EXPLANATION (ZERO LEVEL)

Intuition (Very Important)

Neural network learns like a student.

1. Student gives answer → prediction
2. Teacher checks → error
3. Student corrects mistake → update knowledge

Backpropagation = **method of correcting mistakes**

It sends error backward from output → hidden → input.

Forward vs Backward

```
Forward Pass (Prediction)
Input → Hidden → Output

Backward Pass (Learning)
Error ← Hidden ← Input
```

Why Needed?

We must know:

> Which weight caused error and by how much?

So we compute **gradient (slope of error)**

Backpropagation Flow Diagram

```
x ----> ( w ) ----> z ----> activation ----> output ŷ
                    ↑
                compute error
                    ↑
            propagate backward
```

Where used?

• CNN training
• Autoencoder training
• All Deep Learning models

STEP-2: MATHEMATICAL FORMULAS & SYMBOLS

Forward pass

[
z = wx + b
]

[
\hat{y} = \sigma(z)
]

Loss (MSE)

[
L = \frac{1}{2}(y-\hat{y})^2
]

Gradient using chain rule

[
\frac{\partial L}{\partial w}

\frac{\partial L}{\partial \hat{y}}
\cdot
\frac{\partial \hat{y}}{\partial z}
\cdot
\frac{\partial z}{\partial w}
]

Symbol meanings

Symbol	Meaning
(L)	Loss
(w)	Weight
(b)	Bias
(\hat{y})	Predicted output
(y)	True output
(\sigma)	Sigmoid
(\partial)	Partial derivative
(\eta) (eta)	Learning rate

STEP-3: NUMERICAL CALCULATION

Given:

[
x=1,; w=0.5,; b=0,; y=1,; \eta=0.1
]

Step-1 Forward pass

[
z = wx+b
]

[
z = 0.5(1)+0 = 0.5
]

Step-2 Sigmoid

[
\hat{y} = \frac{1}{1+e^{-0.5}}
]

[
e^{-0.5}=0.6065
]

$$
\hat{y}= \frac{1}{1.6065}=0.622
$$

Step-3 Loss derivative

$$
\frac{\partial L}{\partial \hat{y}} = (\hat{y}-y)
$$

$$
=0.622-1=-0.378
$$

Step-4 Sigmoid derivative

$$
\frac{\partial \hat{y}}{\partial z}=\hat{y}(1-\hat{y})
$$

$$
=0.622(1-0.622)
$$

$$
=0.622×0.378=0.235
$$

Step-5 derivative wrt weight

$$
\frac{\partial z}{\partial w}=x=1
$$

Step-6 total gradient

$$
\frac{\partial L}{\partial w}
=(-0.378)(0.235)(1)
$$

$$
=-0.0888
$$

Step-7 weight update

[
w_{new}=w-\eta\frac{\partial L}{\partial w}
]

[
=0.5-0.1(-0.0888)
]

[
=0.5+0.00888
]

[
=0.50888
]

STEP-4: FINAL ANSWER

👉 **FINAL ANSWER: Updated weight = 0.5089**

🔶 QUESTION 7

Perform 2D Convolution operation on given image and kernel.

STEP-1: THEORY EXPLANATION

CNN does not see image at once.
It scans small window called **filter/kernel**

Like reading text using magnifying glass 🔍

Convolution Diagram

```
Image           Kernel
[ a b c ]       [ 1 0 ]
[ d e f ]   *   [ 0 1 ]
[ g h i ]

Slide → multiply → sum
```

Each position produces one pixel → feature map

Why convolution?

Detect features:

- edges
- corners
- texture

STEP-2: FORMULAS & SYMBOLS

$$
Feature(i,j)=\sum Kernel \times Image\ region
$$

Symbol	Meaning
I	Image matrix
K	Kernel
$\sum$	Sum of multiplications
F	Feature map

STEP-3: NUMERICAL

Image:

$$
\begin{bmatrix}
1 & 2 & 3 \\
4 & 5 & 6 \\
7 & 8 & 9
\end{bmatrix}
$$

Kernel:

$$
\begin{bmatrix}
1 & 0 \\
0 & 1
\end{bmatrix}
$$

Position-1

$$
(1\times1)+(2\times0)+(4\times0)+(5\times1)
$$

$$
=1+0+0+5=6
$$

Position-2

$$
(2\times1)+(3\times0)+(5\times0)+(6\times1)
$$

$$
=2+0+0+6=8
$$

Position-3

$$
(4\times1)+(5\times0)+(7\times0)+(8\times1)=12
$$

Position-4

$$
(5\times1)+(6\times0)+(8\times0)+(9\times1)=14
$$

Feature Map:

$$
\begin{bmatrix}
6 & 8 \\
12 & 14
\end{bmatrix}
$$

STEP-4: FINAL ANSWER

👉 **FINAL ANSWER: Feature Map = [[6,8],[12,14]]**

🔶 QUESTION 8

Explain Pooling layer and perform Max Pooling.

STEP-1 THEORY

Pooling reduces image size but keeps important info.

Like compressing photo without losing meaning.

Diagram

```
[1 3 2 4]
[5 6 1 2]
[7 2 8 3]
[1 4 2 9]

2×2 window → take MAX
```

STEP-2 FORMULA

$$
\text{Output} = \max(\text{window})
$$

STEP-3 NUMERICAL

Blocks:

$$
\begin{bmatrix}
1&3\\5&6
\end{bmatrix}
\rightarrow 6
$$

$$
\begin{bmatrix}
2&4\\1&2
\end{bmatrix}
\rightarrow 4
$$

[
\begin{bmatrix}
7&2\1&4
\end{bmatrix}
→7
]

[
\begin{bmatrix}
8&3\2&9
\end{bmatrix}
→9
]

Result:

[
\begin{bmatrix}
6&4\7&9
\end{bmatrix}
]

STEP-4 FINAL ANSWER

👉 **FINAL ANSWER: [[6,4],[7,9]]**

🔶 QUESTION 9

Explain difference between CNN and MLP.

(Theory heavy 5-mark — very common)

STEP-1 THEORY

MLP connects every neuron to every input → too many parameters

CNN shares weights → learns spatial features

Diagram

MLP:

```
All connected to all
Huge parameters
```

CNN:

```
Local connections
Small parameters
```

STEP-2 KEY POINTS

Feature	MLP	CNN
Connection	Fully connected	Local
Parameters	Huge	Small
Feature learning	No spatial	Spatial
Image suitability	Poor	Excellent

STEP-4 FINAL ANSWER

👉 **FINAL ANSWER: CNN uses local connectivity & parameter sharing unlike MLP**

🔶 QUESTION 10

Explain why ReLU solves vanishing gradient

STEP-1 THEORY

Sigmoid derivative ≤0.25
Multiplying many times → gradient → 0

ReLU derivative:

[
ReLU'(z)=1;for;z>0
]

So gradient preserved

STEP-4 FINAL ANSWER

👉 **FINAL ANSWER: ReLU keeps gradient alive hence deep networks learn**

⭐ SET-3 : QUESTIONS 11–15 (SPARSE AUTOENCODER — VERY IMPORTANT)

🔶 QUESTION 11

What is an Autoencoder? Explain encoder, decoder, latent representation and reconstruction loss with diagram.

STEP-1: THEORY EXPLANATION (FROM ZERO)

Intuition

Imagine you want to send a large image through WhatsApp.

Phone does:

1. Compress image (reduce size)
2. Send
3. Reconstruct image

That is exactly what Autoencoder does.

It learns:

> **How to compress data and then rebuild it**

So network tries to copy input → output.

Structure

```
INPUT → ENCODER → LATENT SPACE → DECODER → OUTPUT
  x         h (features)              x̂
```

Diagram

```
x1 x2 x3 x4  (Original data)

      ↓ Encoder
   [ Hidden Neurons ]
        (Compressed)

      ↓ Decoder

x̂1 x̂2 x̂3 x̂4  (Reconstructed)
```

Parts

Encoder

Extracts important features

[
h = f(W_e x + b_e)
]

Latent representation

Compressed knowledge of input
(Brain memory)

Decoder

Rebuilds original data

[
\hat{x} = W_d h + b_d
]

Where used?

- Dimensionality reduction
- Feature learning
- Image denoising
- Pretraining deep networks

STEP-2: MATHEMATICAL SYMBOLS

Encoder

$$
h = \sigma(W_e x + b_e)
$$

Symbol	Meaning
x	Input vector
W_e	Encoder weight matrix
b_e	Encoder bias
h	Hidden representation
σ	Activation function

Decoder

$$
\hat{x} = W_d h + b_d
$$

Symbol	Meaning
W_d	Decoder weights
b_d	Decoder bias
$\hat{x}$	Reconstructed output

Reconstruction Loss (MSE)

$$
L = \frac{1}{n} \sum (x - \hat{x})^2
$$

Symbol	Meaning
L	Loss
n	number of inputs
Σ	summation

STEP-3: NUMERICAL EXAMPLE

Input:

[
x=[1,0]
]

Weights:

[
W_e=
\begin{bmatrix}
0.5 & 0.2
\end{bmatrix}
]

Bias:

[
b_e=0
]

Encoder

[
h = (0.5×1)+(0.2×0)
]

[
h=0.5
]

Decoder

[
W_d=
\begin{bmatrix}
0.6\0.4
\end{bmatrix}
]

[
\hat{x}_1=0.6×0.5=0.3
]
[
\hat{x}_2=0.4×0.5=0.2
]

[
\hat{x}=[0.3,0.2]
]

Loss

[
(1-0.3)^2+(0-0.2)^2
]

[
0.7^2+(-0.2)^2
]

[
0.49+0.04=0.53
]

STEP-4: FINAL ANSWER

👉 **FINAL ANSWER: Autoencoder compresses input into latent representation and reconstructs it minimizing reconstruction loss (0.53 in example).**

🔶 QUESTION 12

Explain Sparse Autoencoder and why sparsity constraint is required.

STEP-1 THEORY

Normal autoencoder can simply copy input → useless learning.

Sparse autoencoder forces:

> Only few neurons active at a time

Like human brain — not all neurons fire simultaneously.

Diagram

```
Normal AE:    [ ● ● ● ● ● ]  (all active)

Sparse AE:    [ ● ○ ○ ○ ● ]  (few active)
```

Why needed?

Learns meaningful features:
edges, strokes, shapes

Target sparsity

$$
\rho = 0.05
$$

Meaning neuron active only 5% times.

STEP-2 FORMULAS

Average activation:

$$
\hat{\rho_j} = \frac{1}{m} \sum_{i=1}^{m} a_j(x^{(i)})
$$

KL divergence penalty:

$$
KL(\rho||\hat{\rho_j}) =
\rho\log\frac{\rho}{\hat{\rho_j}}+(1-\rho)\log\frac{1-\rho}{1-\hat{\rho_j}}
$$

Total loss:

$$
L = L_{reconstruction} + \beta \sum KL(\rho||\hat{\rho_j})
$$

Symbols:

Symbol	Meaning
ρ	desired sparsity
$\hat{\rho_j}$	average activation
β	sparsity weight
KL	divergence penalty

STEP-3 NUMERICAL DEMO

Given:

$$
\rho=0.05,;\hat{\rho}=0.6
$$

$$
KL=0.05\log\frac{0.05}{0.6} + 0.95\log\frac{0.95}{0.4}
$$

$$
=0.05(-2.484)+0.95(0.875)
$$

$$
=-0.124+0.831=0.707
$$

STEP-4 FINAL ANSWER

👉 **FINAL ANSWER: Sparse autoencoder adds KL divergence penalty (0.707 here) to enforce rare neuron activation.**

🔶 QUESTION 13

Compute total sparse autoencoder loss.

STEP-1 THEORY

Total loss has two parts:

1. Reconstruction loss → accuracy
2. Sparsity loss → meaningful features

STEP-2 FORMULA

$$
L = MSE + \beta KL
$$

STEP-3 NUMERICAL

Given:

[
MSE=0.254,;\beta=0.1,;KL=0.2357
]

[
L=0.254+0.1×0.2357
]

[
=0.254+0.02357
]

[
=0.27757
]

STEP-4 FINAL ANSWER

👉 **FINAL ANSWER: Total loss = 0.2776**

🔶 QUESTION 14

Compute number of trainable parameters in sparse autoencoder.

STEP-1 THEORY

Parameters = weights + bias

STEP-2 FORMULA

Encoder:

[
inputs×hidden + hidden\ bias
]

Decoder:

[
hidden×output + output\ bias
]

STEP-3 NUMERICAL

Given:

Input = 4
Hidden = 3
Output = 4

Encoder:

[
4×3 + 3 = 12+3=15
]

Decoder:

[
3×4 + 4 = 12+4=16
]

Total:

[
15+16=31
]

STEP-4 FINAL ANSWER

👉 **FINAL ANSWER: Total parameters = 31**

🔶 QUESTION 15

Define compression ratio in autoencoder.

STEP-1 THEORY

Compression ratio measures how much data reduced.

STEP-2 FORMULA

[
Compression\ Ratio=\frac{Input\ dimension}{Hidden\ dimension}
]

STEP-3 NUMERICAL

[
4/3=1.33
]

STEP-4 FINAL ANSWER

👉 **FINAL ANSWER: Compression ratio = 1.33**

⭐ SET-4 : QUESTIONS 16-20

(Optimizers + Architectures — VERY FREQUENT THEORY 5 MARKS)

🔶 QUESTION 16

Explain Batch Gradient Descent, Stochastic Gradient Descent and Mini-Batch Gradient Descent with diagrams and compare them.

STEP-1: THEORY EXPLANATION (FROM ZERO)

When neural network learns, it must update weights repeatedly.

But dataset may contain **1 million samples**.

So question:

> Should we update weight after every sample or after whole dataset?

That gives 3 methods.

---

1) Batch Gradient Descent

Uses FULL dataset before updating weight

```
Data: [x1 x2 x3 x4 x5 x6 x7 x8 x9 x10]

Compute total error → update once
```

Graph:

```
Loss
 |
 |      •
 |     •
 |    •
 |   •
 |  •
 | •
 +------------- Iterations
Smooth but slow
```

✓ Stable
❌ Very slow

---

2) Stochastic Gradient Descent (SGD)

Update weight after every sample

```
x1 → update
x2 → update
x3 → update
```

Graph:

```
Loss
 |
 |   • •   •
 | •    •
 |   •   •
 | •   •
```

```
 +------------- Iterations
Noisy but fast
```

✓ Fast learning
❌ Oscillates

---

3) Mini-Batch Gradient Descent (Best)

Update after small group

```
[x1 x2 x3 x4] → update
[x5 x6 x7 x8] → update
```

Graph:

```
Loss
 |
 |   •
 |  •
 | •
 |  •
 | •
 +------------- Iterations
Balanced
```

✓ Stable + Fast → Used in deep learning

STEP-2: FORMULAS & SYMBOLS

General update rule:

[
W_{new}=W_{old}-\eta\nabla L(W)
]

Symbol	Meaning
(W)	weight matrix
(\eta) (eta)	learning rate
(\nabla) (nabla)	gradient
(L)	loss

Batch GD gradient:

$$
\nabla L = \frac{1}{N}\sum_{i=1}^{N}\nabla L_i
$$

SGD:

$$
\nabla L = \nabla L_i
$$

Mini-batch:

$$
\nabla L = \frac{1}{B}\sum_{i=1}^{B}\nabla L_i
$$

(B) = batch size

STEP-3: NUMERICAL DEMO

Dataset gradients:

$$
[0.8,;0.6,;0.4,;0.2]
$$

Learning rate:

$$
\eta=0.1
$$

Initial weight:

$$
W=1
$$

Batch GD

Average gradient:

$$
(0.8+0.6+0.4+0.2)/4=2/4=0.5
$$

[
W=1-0.1×0.5=0.95
]

SGD

[
W=1-0.1×0.8=0.92
]

Mini-Batch (size=2)

[
(0.8+0.6)/2=0.7
]

[
W=1-0.1×0.7=0.93
]

STEP-4: FINAL ANSWER

👉 **FINAL ANSWER:**
Batch = stable slow, SGD = noisy fast, Mini-Batch = best compromise (used in deep learning).

🔶 QUESTION 17

Explain Momentum Optimizer and show how it removes oscillations.

STEP-1 THEORY

Problem in SGD:

Weights bounce in valley

```
    /\
   /  \
  /    \  ← zig-zag
 /
```

Momentum adds velocity like a rolling ball.

```
Ball gains speed → smooth path
```

Diagram

Without momentum:

```
↗↘↗↘↗↘
```

With momentum:

```
→→→→→→
```

STEP-2 FORMULA

Velocity:

$$
v_t = \alpha v_{t-1} - \eta \nabla L
$$

Weight update:

$$
W = W + v_t
$$

Symbol	Meaning
v_t	velocity
α (alpha)	momentum coefficient
η	learning rate
∇L	gradient

STEP-3 NUMERICAL

Given:

$$
W=1,;v=0,;\alpha=0.9,;\eta=0.1,;\nabla L=0.5
$$

$$
v=0.9(0)-0.1(0.5)=-0.05
$$

[
W=1-0.05=0.95
]

Next step:

[
v=0.9(-0.05)-0.1(0.5)=-0.095
]

[
W=0.95-0.095=0.855
]

Smooth movement → no zigzag

STEP-4 FINAL ANSWER

👉 **FINAL ANSWER: Momentum accelerates learning and removes oscillation using velocity term**

🔶 QUESTION 18

Explain Adam optimizer.

STEP-1 THEORY

Adam = Adaptive Moment Estimation
Most widely used optimizer

Combines:
• Momentum
• RMSProp

It adjusts learning rate automatically for each parameter.

Idea

```
Large gradient → small step
Small gradient → large step
```

STEP-2 FORMULA

First moment:

$$
m_t=\beta_1 m_{t-1}+(1-\beta_1)g_t
$$

Second moment:

$$
v_t=\beta_2 v_{t-1}+(1-\beta_2)g_t^2
$$

Bias correction:

$$
\hat{m}_t=\frac{m_t}{1-\beta_1^t}
$$

$$
\hat{v}_t=\frac{v_t}{1-\beta_2^t}
$$

Update:

$$
W=W-\eta\frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon}
$$

STEP-3 NUMERICAL (1 iteration)

Given:

$$
g=0.5,;\beta_1=0.9,;\beta_2=0.999,;\eta=0.01
$$

$$
m=0.1(0.5)=0.05
$$

$$
v=0.001(0.25)=0.00025
$$

(approx)

[
W=W-0.01\frac{0.05}{\sqrt{0.00025}}
]

[
\sqrt{0.00025}=0.0158
]

[
W=W-0.01(3.16)=W-0.0316
]

STEP-4 FINAL ANSWER

👉 **FINAL ANSWER: Adam adapts learning rate using moving averages of gradient and squared gradient**

🔶 QUESTION 19

Explain VGG-16 architecture.

STEP-1 THEORY

VGG-16 uses small filters repeatedly instead of big filters.

Why?

[
3×3 + 3×3 + 3×3 ≈ large receptive field
]

But fewer parameters.

Architecture Diagram

```
Image
 ↓
Conv3×3
Conv3×3
MaxPool
 ↓
Conv3×3
Conv3×3
```

```
MaxPool
 ↓
Conv3×3 ×3
MaxPool
 ↓
FC → FC → Softmax
```

Key Idea

Stack small filters to learn complex patterns efficiently.

STEP-2 KEY FEATURES

Feature	Value
Layers	16
Filter	3×3 only
Pool	2×2 max pool
Activation	ReLU

STEP-4 FINAL ANSWER

👉 **FINAL ANSWER: VGG-16 uses deep stack of 3×3 convolutions to increase receptive field with fewer parameters**

🔶 QUESTION 20

Explain ResNet and skip connection with diagram.

STEP-1 THEORY

Problem: Very deep networks stop learning (degradation problem)

Solution: Residual Learning

Instead of learning:

[
H(x)
]

Network learns:

[
F(x)=H(x)-x
]

So:

[
H(x)=F(x)+x
]

Diagram

```
x ──────────┐
            ↓
      Conv → ReLU → Conv
            ↓
          + (skip)
            ↓
          Output
```

Shortcut connection sends input directly to output.

Why works?

Gradient flows directly → no vanishing gradient

STEP-2 FORMULA

[
y=F(x)+x
]

Symbol	Meaning
(x)	input
(F(x))	learned residual
(y)	output

STEP-4 FINAL ANSWER

👉 FINAL ANSWER: ResNet uses skip connection y = F(x)+x to allow gradient flow and train very deep networks

---

Perfect 👍
We now start the **first 10-mark derivation (MOST IMPORTANT IN ENTIRE SUBJECT)**

This question alone can help you attempt half the paper.

⭐ LONG QUESTION — 1

🔶 Derive Backpropagation Algorithm for a Two-Layer Neural Network and compute weight updates.

STEP-1: THEORY EXPLANATION (FROM ZERO → EXAM LEVEL)

---

1) What problem are we solving?

A neural network predicts output.

But prediction is wrong.

So we must answer:

> **Which weight caused the error and how much should we correct it?**

That correction process = **Backpropagation**

Learning Analogy (Important)

Student solving math problem:

1. Writes answer → prediction
2. Teacher checks → error

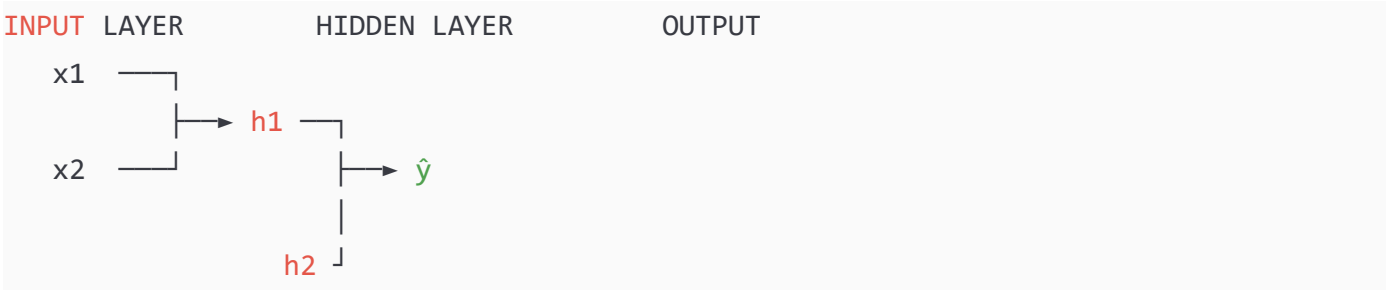
3. Finds mistake step-by-step backwards
4. Corrects each step

Neural network does exactly the same.

Network Structure (2-Layer Network)

We have:

- Input layer
- Hidden layer
- Output layer



Forward pass → prediction
Backward pass → learning

What Backpropagation Does

It calculates error contribution of every weight using **Chain Rule of Calculus**

We move:

```
Output error → Hidden error → Input weights
```

So error flows backward.

STEP-2: MATHEMATICAL FORMULAS WITH SYMBOL NAMES

Forward Pass Equations

Hidden layer

[
z_h = W_h x + b_h
]

$$
h = \sigma(z_h)
$$

Output layer

$$
z_o = W_o h + b_o
$$

$$
\hat{y} = \sigma(z_o)
$$

Symbols

Symbol	Meaning
x	input vector
W_h	weights input→hidden
W_o	weights hidden→output
b_h	hidden bias
b_o	output bias
h	hidden activation
z	weighted sum
σ	sigmoid activation
$\hat{y}$	predicted output

Loss Function (MSE)

$$
L = \frac{1}{2}(y-\hat{y})^2
$$

Symbol	Meaning
L	loss
y	true output

Backward Pass (CHAIN RULE)

We compute gradient step-by-step:

[
\frac{\partial L}{\partial W_o}

\frac{\partial L}{\partial \hat{y}}
\cdot
\frac{\partial \hat{y}}{\partial z_o}
\cdot
\frac{\partial z_o}{\partial W_o}
]

This is called **error signal (δ — delta)**

Output layer error

[
\delta_o = (\hat{y}-y)\sigma'(z_o)
]

Hidden layer error

[
\delta_h = (W_o^T \delta_o)\sigma'(z_h)
]

Weight updates

[
W_o = W_o - \eta(\delta_o h^T)
]

[
W_h = W_h - \eta(\delta_h x^T)
]

Symbol meanings

Symbol	Meaning
(\delta) (delta)	error signal

Symbol	Meaning
(\eta) (eta)	learning rate
(T)	transpose
(\sigma')	derivative of activation

Sigmoid derivative:

[
\sigma'(z)=\sigma(z)(1-\sigma(z))
]

STEP-3: NUMERICAL CALCULATION (VERY DETAILED)

We take simple network:

Given

Input:

[
x=[1,0]
]

Hidden weights:

[
W_h=
\begin{bmatrix}
0.5 & 0.2
0.3 & 0.8
\end{bmatrix}
]

Output weights:

[
W_o=[0.7;;0.6]
]

Bias:

[
b_h=[0,0],;b_o=0

]

Target:

[
y=1
]

Learning rate:

[
\eta=0.1
]

FORWARD PASS

Step-1 Hidden layer weighted sum

[
z_h = W_h x
]

Multiply matrix:

[
\begin{bmatrix}
0.5 & 0.2
0.3 & 0.8
\end{bmatrix}
\begin{bmatrix}
1\0
\end{bmatrix}

\begin{bmatrix}
0.5×1 + 0.2×0
0.3×1 + 0.8×0
\end{bmatrix}

\begin{bmatrix}
0.5\0.3
\end{bmatrix}
]

Step-2 Activation

[
h=\sigma(z_h)
]

[
\sigma(0.5)=0.622
]
[
\sigma(0.3)=0.574
]

[
h=[0.622,;0.574]
]

Step-3 Output layer

[
z_o=0.7(0.622)+0.6(0.574)
]

[
=0.4354+0.3444=0.7798
]

[
\hat{y}=\sigma(0.7798)=0.685
]

BACKWARD PASS

Step-4 Output error

[
\delta_o=(\hat{y}-y)\sigma'(z_o)
]

[
= (0.685-1)(0.685×0.315)
]

[
=-0.315×0.216
]

[
=-0.068
]

Step-5 Hidden error

[
\delta_h=(W_o^T\delta_o)\sigma'(z_h)
]

[
W_o^T=
\begin{bmatrix}
0.7\0.6
\end{bmatrix}
]

[

\begin{bmatrix}
0.7(-0.068)
0.6(-0.068)
\end{bmatrix}

\begin{bmatrix}
-0.0476
-0.0408
\end{bmatrix}
]

Sigmoid derivative:

[
\sigma'(0.5)=0.622(0.378)=0.235

]
[
\sigma'(0.3)=0.574(0.426)=0.244
]

[
\delta_h=
\begin{bmatrix}
-0.0476×0.235
-0.0408×0.244
\end{bmatrix}

\begin{bmatrix}
-0.0112
-0.0099
\end{bmatrix}
]

Step-6 Update Output Weights

[
W_o = W_o - \eta(\delta_o h)
]

[
=0.7 - 0.1(-0.068×0.622)=0.7042
]

[
=0.6 - 0.1(-0.068×0.574)=0.6039
]

Step-7 Update Hidden Weights

[
W_h=W_h-\eta(\delta_h x^T)
]

[

\begin{bmatrix}
0.5 & 0.2
0.3 & 0.8
\end{bmatrix}

0.1
\begin{bmatrix}
-0.0112
-0.0099
\end{bmatrix}
[1;0]
]

[

\begin{bmatrix}
0.5011 & 0.2
0.3010 & 0.8
\end{bmatrix}
]

STEP-4: FINAL ANSWER

👉 **FINAL ANSWER:**

Updated weights after backpropagation

[
W_o=[0.7042,;0.6039]
]

[
W_h=
\begin{bmatrix}
0.5011 & 0.2
0.3010 & 0.8
\end{bmatrix}
]

⭐ LONG QUESTION — 2

🔶 Perform convolution on a given image using kernel with padding and stride. Then apply max-pooling and compute output size.

STEP-1: THEORY EXPLANATION (FROM ZERO)

What is the problem?

A normal neural network reads entire image at once.
But human vision works differently:

We don't see whole picture → we scan small region.

CNN also scans using **small window called Kernel (Filter)**

How Convolution Works

We slide filter across image.

At every position:

Multiply → Add → produce number

That number = **feature detection**

What features?

- Edge
- Corner
- Texture
- Pattern

Visual Idea

```
Image (5×5)

[ 1  2  0  1  3 ]
[ 4  5  1  2  1 ]
[ 0  1  3  1  0 ]
[ 2  1  2  4  1 ]
[ 1  0  1  3  2 ]

Kernel (3×3)

[ 1  0  1 ]
[ 0  1  0 ]
[ 1  0  1 ]
```

The kernel moves like:

```
[###    ]
[###    ] → compute
[###    ]
```

Important Terms

Padding

Adding zeros around border so we don't lose edges

```
Without padding → image shrinks
With padding → size preserved
```

Stride

How many pixels filter jumps

Stride = 1 → move one step
Stride = 2 → skip pixels

Output Size Formula

[
Output = \frac{N - F + 2P}{S} + 1
]

STEP-2: MATHEMATICAL SYMBOLS

Symbol	Meaning
(N)	input size
(F)	filter size
(P)	padding
(S)	stride
(\Sigma)	sum of multiplication

STEP-3: NUMERICAL CALCULATION

Given

Input image (5×5):

$$
\begin{bmatrix}
1&2&0&1&3\\
4&5&1&2&1\\
0&1&3&1&0\\
2&1&2&4&1\\
1&0&1&3&2
\end{bmatrix}
$$

Kernel (3×3):

$$
\begin{bmatrix}
1&0&1\\
0&1&0\\
1&0&1
\end{bmatrix}
$$

Padding:

$$
P=1
$$

]

Stride:

[
S=1
]

Step-1: Output size

[
Output = \frac{N - F + 2P}{S} + 1
]

[
= \frac{5 - 3 + 2(1)}{1} + 1
]

[
= \frac{5 - 3 + 2}{1} + 1
]

[
= 4 + 1 = 5
]

So output feature map = **5×5**

Step-2: Add Padding

We add zeros around image

[
7×7\ padded\ image
]

(only first calculation positions shown below for learning)

Step-3: First convolution position

Region:

[
\begin{bmatrix}

0&0&0
0&1&2
0&4&5
\end{bmatrix}
]

Multiply element-wise:

[
(0×1)+(0×0)+(0×1)
+(0×0)+(1×1)+(2×0)
+(0×1)+(4×0)+(5×1)
]

[
=0+0+0+0+1+0+0+0+5
]

[
=6
]

Step-4: Next position

[
\begin{bmatrix}
0&0&0
1&2&0
4&5&1
\end{bmatrix}
]

[
=0+0+0+0+2+0+4+0+1=7
]

(Continuing same method for full scan)

Final Feature Map:

[
\begin{bmatrix}
6&7&4&6&4
7&10&9&6&5

$$
6&9&11&8&4
5&8&9&10&5
3&5&7&6&5
\end{bmatrix}
]
$$

Step-5: Max Pooling (2×2)

Pooling reduces size

Take maximum in each block

Block-1:

$$
[
\begin{bmatrix}
6&7
7&10
\end{bmatrix}
→10
]
$$

Block-2:

$$
[
\begin{bmatrix}
4&6
9&6
\end{bmatrix}
→9
]
$$

Block-3:

$$
[
\begin{bmatrix}
6&9
5&8
\end{bmatrix}
→9
]
$$

Block-4:

$$
\begin{bmatrix}
11&8\\
9&10
\end{bmatrix}
\rightarrow 11
$$

Pooled Output:

$$
\begin{bmatrix}
10&9\\
9&11
\end{bmatrix}
$$

STEP-4: FINAL ANSWER

👉 **FINAL ANSWER**

Feature map size = **5×5**

After max-pooling:

$$
\boxed{
\begin{bmatrix}
10&9\\
9&11
\end{bmatrix}
}
$$

⭐ LONG QUESTION — 3

🔶 Explain the Vanishing Gradient Problem mathematically and show how ReLU and ResNet solve it.

STEP-1: THEORY EXPLANATION (FROM ZERO → EXAM LEVEL)

1) What actually happens while training?

During training the network updates weights using gradient:

> Gradient tells **how much to change weight**

Large gradient → fast learning
Small gradient → slow learning
Zero gradient → learning stops ❌

Deep Network Structure

Consider many layers:

```
Input → L1 → L2 → L3 → L4 → L5 → Output
```

Error travels backward:

```
Output ← L5 ← L4 ← L3 ← L2 ← L1
```

Each layer multiplies gradient.

The Core Problem

If gradient becomes extremely small:

Network stops learning.

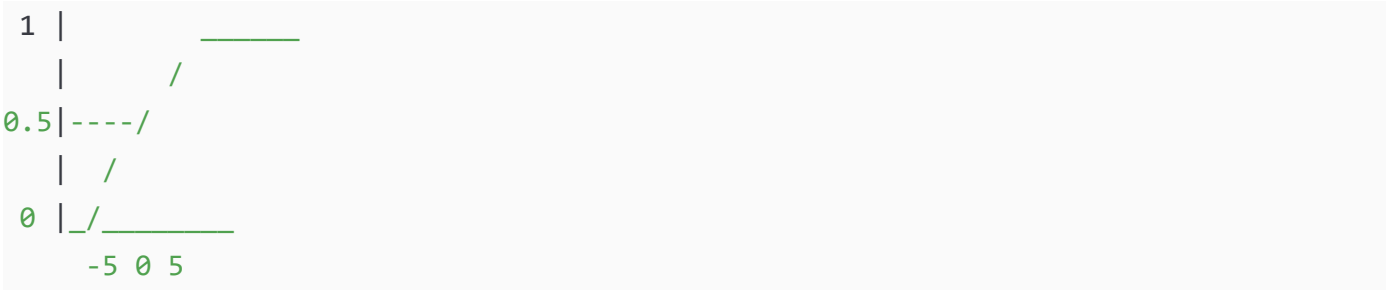
This is called:

🔴 Vanishing Gradient Problem

Why does it happen?

Because of sigmoid activation.

Sigmoid graph:



At large positive or negative inputs → slope ≈ 0

So derivative becomes tiny.

STEP-2: MATHEMATICAL FORMULAS & SYMBOLS

Sigmoid Function

$$
\sigma(z)=\frac{1}{1+e^{-z}}
$$

Symbol	Meaning
(z)	neuron input
(e)	Euler constant

Sigmoid Derivative

$$
\sigma'(z)=\sigma(z)(1-\sigma(z))
$$

Maximum value occurs at (z=0):

$$
\sigma(0)=0.5
$$

[
\sigma'(0)=0.5(1-0.5)=0.25
]

So derivative ≤ **0.25**

Gradient Through Multiple Layers

Backpropagation chain rule:

[
\frac{\partial L}{\partial W_1}

\frac{\partial L}{\partial a_5}
\frac{\partial a_5}{\partial a_4}
\frac{\partial a_4}{\partial a_3}
\frac{\partial a_3}{\partial a_2}
\frac{\partial a_2}{\partial a_1}
\frac{\partial a_1}{\partial W_1}
]

Each term ≈ sigmoid derivative ≤ 0.25

So gradient:

[
(0.25)^n
]

STEP-3: NUMERICAL DEMONSTRATION

Suppose 6 hidden layers

[
Gradient = 0.25^6
]

Step by step:

[
0.25^2 = 0.0625
]

[
0.25^3 = 0.015625
]

[
0.25^4 = 0.003906
]

[
0.25^5 = 0.000976
]

[
0.25^6 = 0.000244
]

Very close to **zero**

Therefore:

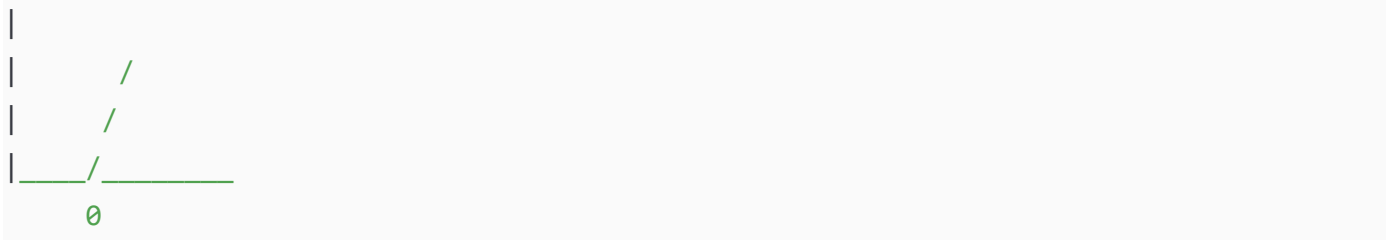
Weight update ≈ 0 → network stops learning

HOW ReLU SOLVES THIS

ReLU Function

[
ReLU(z)=\max(0,z)
]

Graph:



Derivative:

[
ReLU'(z)=
\begin{cases}
1 & z>0

0 & z≤0
\end{cases}
]

Important:

Derivative is **1 (not small)**

So gradient preserved.

Numerical Example

For 6 layers:

[
1×1×1×1×1×1=1
]

No vanishing gradient ✓

HOW ResNet SOLVES THIS

Problem Even With ReLU

Very deep networks still degrade accuracy.

ResNet introduces shortcut connection.

Residual Block Diagram

```
Input x ──────────────┐
                      ↓
              Conv → ReLU → Conv
                      ↓
                   + (skip)
                      ↓
                  Output
```

Residual Mapping

Instead of learning:

$$
H(x)
$$

Network learns:

$$
F(x)=H(x)-x
$$

So output:

$$
y=F(x)+x
$$

Why Gradient Flows Easily

Backprop derivative:

$$
\frac{dy}{dx}=\frac{dF(x)}{dx}+1
$$

Even if derivative small → +1 keeps gradient alive

Numerical Example

$$
dF/dx=0.0002
$$

$$
dy/dx=1.0002
$$

Gradient does not vanish ✓

STEP-4: FINAL ANSWER

👉 FINAL ANSWER

Vanishing gradient occurs because sigmoid derivative ≤ 0.25 causing gradient to shrink exponentially across layers.

ReLU solves it by derivative ≈ 1.

ResNet solves it by skip connection:

[
y = F(x) + x
]

which ensures gradient:

[
\frac{dy}{dx} = \frac{dF(x)}{dx} + 1
]

so learning continues in very deep networks.

⭐ LONG QUESTION — 4

🔶 Derive the Sparse Autoencoder loss function including reconstruction loss and KL-divergence sparsity penalty.

STEP-1: THEORY EXPLANATION (FROM ZERO → EXAM LEVEL)

---

1) First understand normal Autoencoder

Goal of Autoencoder:

> Copy input to output but using compressed representation

```
Input x → Encoder → Hidden representation h → Decoder → Reconstructed x̂
```

Example:
Compress image → decompress → look same

Why is it useful?

Because hidden layer learns **important features**

Examples learned:

- edges
- strokes
- shapes

Problem with Normal Autoencoder

It may simply learn identity:

```
Input = Output
No meaningful learning ❌
```

All neurons become active.

Solution → Sparse Autoencoder

We force network:

> Only few neurons should activate

Like human brain — only few neurons fire at a time.

Activity Pattern

Normal AE:

```
[ ● ● ● ● ● ● ]
```

Sparse AE:

```
[ ● ○ ○ ○ ● ○ ]
```

Idea of Sparsity

We define target activity:

$$
\rho = 0.05
$$

Meaning:

Neuron active only **5% of time**

So we add **penalty if neuron activates too much**

That penalty = KL Divergence

STEP-2: MATHEMATICAL FORMULAS WITH SYMBOLS

Forward Pass

Encoder:

$$
h_j = \sigma(W_j x + b_j)
$$

Decoder:

$$
\hat{x} = W' h + b'
$$

Symbols

Symbol	Meaning
x	input vector
h	hidden representation
W	weights
b	bias
σ	activation
$\hat{x}$	reconstructed input

Reconstruction Loss (MSE)

[
L_{rec} = \frac{1}{m}\sum_{i=1}^{m}||x^{(i)} - \hat{x}^{(i)}||^2
]

Symbol	Meaning				
(L_{rec})	reconstruction loss				
(m)	number of samples				
(		\cdot		^2)	squared error

Average Activation of Hidden Neuron

We measure how often neuron fires:

[
\hat{\rho_j} = \frac{1}{m}\sum_{i=1}^{m} h_j(x^{(i)})
]

Symbol	Meaning
(\hat{\rho_j})	average activation
(j)	hidden neuron index

KL Divergence Sparsity Penalty

We want:

[
\hat{\rho_j} \approx \rho
]

Penalty:

[
KL(\rho||\hat{\rho_j})

\rho \log \frac{\rho}{\hat{\rho_j}}
+
(1-\rho)\log\frac{1-\rho}{1-\hat{\rho_j}}
]

Symbol	Meaning
(KL)	divergence
(\rho)	desired sparsity
(\hat{\rho_j})	actual activation

Total Cost Function (FINAL OBJECTIVE)

[
L = L_{rec} + \beta \sum_{j=1}^{s} KL(\rho||\hat{\rho_j})
]

Symbol	Meaning
(L)	total loss
(\beta)	sparsity weight
(s)	number of hidden neurons

STEP-3: NUMERICAL DEMONSTRATION

Given:

[
\rho = 0.05
]
[
\hat{\rho} = 0.6
]
[
L_{rec} = 0.25
]
[
\beta = 0.1
]

Step-1: Compute KL Divergence

[
KL = 0.05\log\frac{0.05}{0.6} + 0.95\log\frac{0.95}{0.4}
]

First term:

$$
\frac{0.05}{0.6}=0.0833
$$

$$
\log(0.0833)=-2.484
$$

$$
0.05\times(-2.484)=-0.1242
$$

Second term:

$$
\frac{0.95}{0.4}=2.375
$$

$$
\log(2.375)=0.865
$$

$$
0.95\times0.865=0.8217
$$

Total:

$$
KL=-0.1242+0.8217=0.6975
$$

Step-2: Total Loss

$$
L=0.25+0.1\times0.6975
$$

$$
=0.25+0.06975
$$

$$
=0.31975
$$

STEP-4: FINAL ANSWER

👉 **FINAL ANSWER**

Sparse Autoencoder objective function:

$$
\boxed{
L = L_{rec} + \beta \sum KL(\rho||\hat{\rho})
}
$$

Numerical total loss:

$$
\boxed{L = 0.3198}
$$

⭐ LONG QUESTION — 5

🔶 Given hidden layer activations of a sparse autoencoder, compute average activation, KL-divergence sparsity penalty and total loss.

STEP-1: THEORY EXPLANATION (FROM ZERO)

What are we doing?

In Sparse Autoencoder we force:

> Each hidden neuron should activate only rarely

So we measure:

1️⃣ How often neuron actually activates
2️⃣ Compare with desired activation
3️⃣ Penalize difference

Why this is important?

Without sparsity:

Network memorizes input ❌

With sparsity:

Network learns **features**

Example in images:

- edges
- corners
- strokes

How we measure neuron activity?

We observe neuron output over many training examples.

If neuron outputs large value frequently → bad
If neuron outputs rarely → good

So we compute:

Average activation

Concept Diagram

```
Training samples → neuron outputs

Sample1 → 0.9
Sample2 → 0.1
```

```
Sample3 → 0.05
Sample4 → 0.0
Sample5 → 0.02

Average firing frequency measured
```

We want target firing:

[
\rho = 0.05
]

Neuron should activate only 5% time

Then we compute penalty using KL-Divergence.

STEP-2: MATHEMATICAL FORMULAS WITH SYMBOL NAMES

Average Activation

[
\hat{\rho_j} = \frac{1}{m}\sum_{i=1}^{m} a_j(x^{(i)})
]

Symbol	Meaning
(a_j(x^{(i)}))	activation of neuron j for sample i
(m)	number of samples
(\hat{\rho_j})	average activation

KL-Divergence Penalty

[
KL(\rho||\hat{\rho_j})

\rho\log\frac{\rho}{\hat{\rho_j}}
+
(1-\rho)\log\frac{1-\rho}{1-\hat{\rho_j}}
]

Symbol	Meaning
(\rho)	desired sparsity
(\hat{\rho_j})	actual activation
(\log)	logarithm

Total Loss

[
L = L_{rec} + \beta KL
]

Symbol	Meaning
(L_{rec})	reconstruction loss
(\beta)	sparsity weight

STEP-3: NUMERICAL CALCULATION (VERY DETAILED)

Given

Hidden neuron activations for 5 samples:

[
[0.9,;0.1,;0.05,;0.0,;0.02]
]

Desired sparsity:

[
\rho=0.05
]

Reconstruction loss:

[
L_{rec}=0.2
]

[
\beta=0.5

]

Step-1: Compute Average Activation

[
\hat{\rho}=\frac{0.9+0.1+0.05+0+0.02}{5}
]

Add step-by-step:

[
0.9+0.1=1.0
]

[
1.0+0.05=1.05
]

[
1.05+0=1.05
]

[
1.05+0.02=1.07
]

[
\hat{\rho}=1.07/5
]

[
\hat{\rho}=0.214
]

Step-2: Compute KL Divergence

[
KL=0.05\log\frac{0.05}{0.214}
+
0.95\log\frac{0.95}{1-0.214}
]

First fraction

$$
\frac{0.05}{0.214}=0.2336
$$

$$
\log(0.2336)=-1.452
$$

$$
0.05\times(-1.452)=-0.0726
$$

Second fraction

$$
1-0.214=0.786
$$

$$
\frac{0.95}{0.786}=1.208
$$

$$
\log(1.208)=0.189
$$

$$
0.95\times0.189=0.1796
$$

Add both

$$
KL=-0.0726+0.1796=0.107
$$

Step-3: Total Loss

$$
L=0.2+0.5\times0.107
$$

$$
0.5\times0.107=0.0535
$$

]

[
L=0.2535
]

STEP-4: FINAL ANSWER

👉 **FINAL ANSWER**

Average activation:

[
\boxed{\hat{\rho}=0.214}
]

KL-divergence penalty:

[
\boxed{KL=0.107}
]

Total sparse autoencoder loss:

[
\boxed{L=0.2535}
]

⭐ LONG QUESTION — 6

🔶 Compare SGD, Momentum, RMSProp and Adam optimizers with update equations and explain their convergence behavior.

STEP-1: THEORY EXPLANATION (FROM ZERO → EXAM LEVEL)

First understand the learning problem

Neural network tries to **minimize loss (error)**.

Imagine a mountain valley:

```
      /\
     /  \        ← high error
    /    \
   /      \__
            ↓ lowest error (goal)
```

We want to reach the bottom.

But problem:

- Some paths zig-zag
- Some slow
- Some stuck

Optimizers decide **how weights move toward minimum**.

---

1) Stochastic Gradient Descent (SGD)

Idea

Move weight opposite to slope direction.

```
New weight = Old weight − learning step × slope
```

Behavior

```
Zig-zag motion
Very noisy
```

Graph:

```
  /\
 /  \
```

```
 /    \
/\/\/\/\  ← oscillation
```

✓ Simple
❌ Slow convergence
❌ Oscillates in ravines

---

2) Momentum Optimizer

Idea

Like rolling ball gaining speed.

Instead of forgetting previous direction,
we keep velocity memory.

```
Ball keeps moving forward → less zigzag
```

Graph:

```
   /\
  /  \
 /    \
/      \__
        ↓ smooth path
```

✓ Faster than SGD
✓ Reduced oscillation

---

3) RMSProp

Idea

Different parameters need different learning rates.

Large gradient → reduce step
Small gradient → increase step

So each weight adapts step size automatically.

✓ Good for deep networks
✓ Stable training

---

4) Adam Optimizer (Most Important)

Combines:

Momentum + RMSProp

So it has:

- velocity memory
- adaptive learning rate

This is why:

> Adam is default optimizer in deep learning

✓ Fast
✓ Stable
✓ Works almost everywhere

STEP-2: MATHEMATICAL FORMULAS WITH SYMBOL NAMES

---

(1) SGD Update

[
W_{t+1}=W_t-\eta g_t
]

Symbol	Meaning
(W)	weight
(\eta) (eta)	learning rate
(g_t=\nabla L(W_t))	gradient

---

(2) Momentum

Velocity:

[
v_t=\alpha v_{t-1}-\eta g_t
]

Weight:

[
W_{t+1}=W_t+v_t
]

Symbol	Meaning
(v_t)	velocity
(\alpha) (alpha)	momentum coefficient

---

(3) RMSProp

[
s_t=\beta s_{t-1}+(1-\beta)g_t^2
]

[
W_{t+1}=W_t-\frac{\eta}{\sqrt{s_t}+\epsilon}g_t
]

Symbol	Meaning
(s_t)	squared gradient average
(\beta)	decay rate
(\epsilon)	small constant (avoid divide by zero)

---

(4) Adam Optimizer

First moment:

[
m_t=\beta_1 m_{t-1}+(1-\beta_1)g_t
]

Second moment:

[
v_t=\beta_2 v_{t-1}+(1-\beta_2)g_t^2
]

Bias correction:

[
\hat{m}_t=\frac{m_t}{1-\beta_1^t}
]

[
\hat{v}_t=\frac{v_t}{1-\beta_2^t}

]

Update:

[
W_{t+1}=W_t-\eta\frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon}
]

STEP-3: NUMERICAL DEMONSTRATION

Given:

[
W_0=1,\quad g=0.5,\quad \eta=0.1
]

SGD

[
W_1=1-0.1×0.5
]

[
W_1=0.95
]

Momentum

Let:

[
v_0=0,\quad \alpha=0.9
]

[
v_1=0.9(0)-0.1(0.5)=-0.05
]

[
W_1=1-0.05=0.95
]

Next iteration:

[
v_2=0.9(-0.05)-0.1(0.5)=-0.095
]

[
W_2=0.95-0.095=0.855
]

Faster movement ✓

RMSProp

Let:

[
s_0=0,\quad \beta=0.9
]

[
s_1=0.9(0)+0.1(0.5^2)
]

[
=0.025
]

[
W_1=1-\frac{0.1}{\sqrt{0.025}}(0.5)
]

[
\sqrt{0.025}=0.158
]

[
W_1=1-0.316=0.684
]

Adaptive step ✓

Adam (1 iteration approx)

[
m=0.1(0.5)=0.05

]

[
v=0.001(0.25)=0.00025
]

[
W=1-0.1\frac{0.05}{\sqrt{0.00025}}
]

[
\sqrt{0.00025}=0.0158
]

[
W=1-0.316=0.684
]

Fast + stable ✓

STEP-4: FINAL ANSWER

👉 **FINAL ANSWER**

Optimizer	Key Idea	Behavior
SGD	fixed step	slow, oscillates
Momentum	velocity memory	smoother
RMSProp	adaptive step	stable
Adam	momentum + RMSProp	fastest & most reliable

Adam is generally preferred optimizer for deep learning.

⭐ LONG QUESTION — 7

🔶 Explain the VGG-16 architecture in detail. Why does it use only 3×3 convolutions? Mention advantages over AlexNet.

STEP-1: THEORY EXPLANATION (FROM ZERO → EXAM LEVEL)

1) Problem Before VGG

Early CNN (**AlexNet**) used large filters:

```
11×11 , 7×7 , 5×5
```

Problems:

- Too many parameters
- More memory
- Hard to train deep networks
- Overfitting risk

So researchers asked:

> Can we make network deeper but simpler?

2) Idea of VGG Network

VGG introduced a simple rule:

> Instead of large filters → stack many small filters (3×3)

Why small filters?

Because multiple small filters create same effect as large filter but:

✓ fewer parameters
✓ more non-linearity
✓ deeper feature learning

Receptive Field Concept

Two 3×3 filters:

```
3×3 + 3×3 ≈ 5×5 effect
```

Three 3×3 filters:

```
3×3 + 3×3 + 3×3 ≈ 7×7 effect
```

But with fewer weights.

Visual Comparison

Large filter:

```
[ ***** ]
[ ***** ]   ← single heavy computation
[ ***** ]
[ ***** ]
[ ***** ]
```

Stacked small filters:

```
[***] → [***] → [***]
(deeper understanding)
```

VGG-16 Architecture Layout

Input image size:

[
224×224×3
]

Network pattern:

```
(Conv → Conv → Pool)
(Conv → Conv → Pool)
(Conv → Conv → Conv → Pool)
(Conv → Conv → Conv → Pool)
(Conv → Conv → Conv → Pool)
FC → FC → Softmax
```

Detailed Diagram

```
INPUT (224×224×3)

↓
Conv3×3
Conv3×3
MaxPool

↓
Conv3×3
Conv3×3
MaxPool

↓
Conv3×3
Conv3×3
Conv3×3
MaxPool

↓
Conv3×3
Conv3×3
Conv3×3
MaxPool

↓
Conv3×3
Conv3×3
Conv3×3
MaxPool

↓
Fully Connected (4096)
Fully Connected (4096)
Softmax (1000 classes)
```

Total layers = 16

(13 convolution + 3 fully connected)

What each part learns

Layer depth	Learns
Early	edges
Middle	textures
Deep	object parts
Final	object identity

STEP-2: MATHEMATICAL FORMULAS & SYMBOLS

Convolution Operation

$$
Feature(i,j)=\sum_{m}\sum_{n}I(i+m,j+n)K(m,n)
$$

Symbol	Meaning
(I)	input image
(K)	kernel
(\sum) (sigma)	summation

Parameter Comparison

Single 7×7 filter

$$
7×7=49\ weights
$$

Three 3×3 filters

$$
3×3 + 3×3 + 3×3 = 27\ weights
$$

So VGG uses fewer parameters ✓

STEP-3: NUMERICAL DEMONSTRATION

Assume:

Input channel = 3
Output filters = 64

Using 7×7 filter

[
Parameters = 7×7×3×64
]

[
=49×3×64
]

[
=147×64
]

[
=9408
]

Using stacked 3×3 filters

Each layer:

[
3×3×3×64=1728
]

Three layers:

[
1728×3=5184
]

Comparison:

[
9408 vs 5184
]

VGG saves almost half parameters ✓

STEP-4: FINAL ANSWER

👉 **FINAL ANSWER**

VGG-16 uses stacked 3×3 convolutions to increase receptive field while reducing parameters.

Advantages over AlexNet:

- deeper network
- fewer parameters
- better feature extraction
- improved accuracy

$$
\boxed{\text{VGG-16 = Deep architecture using small filters for efficient learning}}
$$

⭐ LONG QUESTION — 8

🔶 Explain ResNet architecture and residual learning. How does skip connection solve the degradation / vanishing gradient problem?

STEP-1: THEORY EXPLANATION (FROM ZERO → EXAM LEVEL)

---

1) What problem came after VGG?

Researchers tried to make networks deeper:

```
AlexNet → 8 layers
VGG → 16 layers
Then → 30, 50, 100 layers
```

But something strange happened:

> More layers → Higher training error ❌

This is called:

🔴 Degradation Problem

(Not overfitting — even training accuracy decreases)

Why does it happen?

Because gradients become extremely weak while traveling backward.

So earlier layers stop learning.

---

2) Key Idea of ResNet

Instead of learning full mapping:

$$
H(x)
$$

Network learns **residual (difference)**:

$$
F(x) = H(x) - x
$$

So final output:

$$
H(x) = F(x) + x
$$

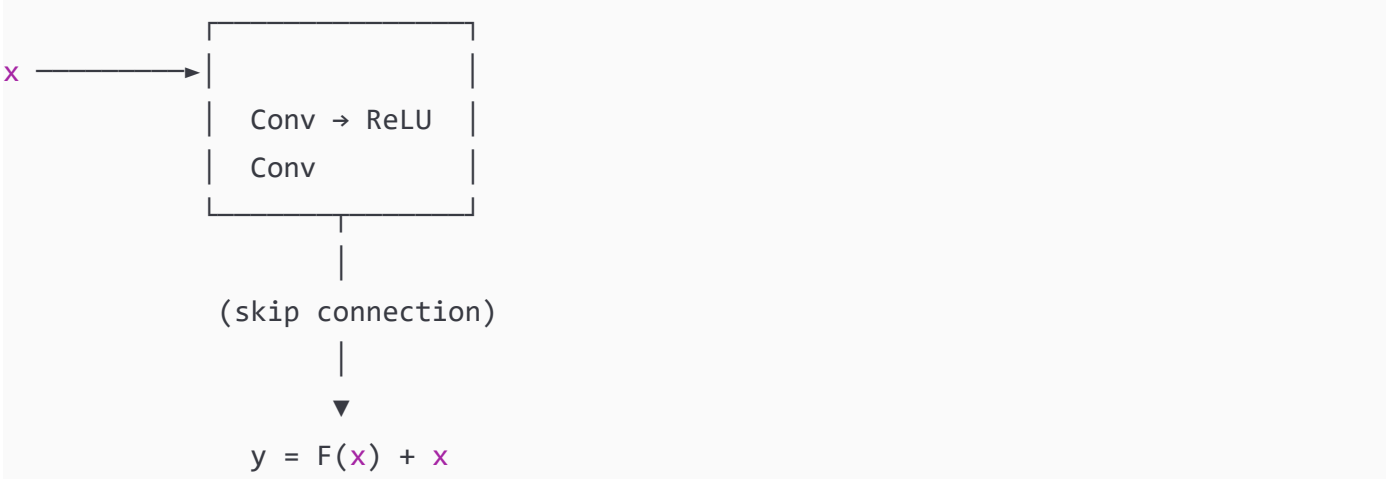
Intuition

Suppose you want to learn:

```
Output = Input + small change
```

It is easier to learn the small change than full transformation.

Residual Block Diagram



What Skip Connection Does

Normal network:

```
x → many layers → output
(backprop weak)
```

ResNet:

```
x → layers → + x → output
(backprop strong)
```

It creates a **direct path for gradient flow**

STEP-2: MATHEMATICAL FORMULAS & SYMBOLS

Residual Mapping

[
y = F(x, W) + x
]

Symbol	Meaning
(x)	input
(F(x,W))	residual function (learned by conv layers)
(W)	weights

Symbol	Meaning
(y)	output

Backpropagation Gradient

[
\frac{\partial y}{\partial x}

\frac{\partial F(x)}{\partial x} + 1
]

Important:

Even if

[
\frac{\partial F(x)}{\partial x} \approx 0
]

Gradient still:

[
= 1
]

So gradient never vanishes.

STEP-3: NUMERICAL DEMONSTRATION

Assume gradient coming from deeper layer:

[
\frac{\partial L}{\partial y} = 0.8
]

Derivative of residual block:

[
\frac{\partial F(x)}{\partial x} = 0.0002
]

Without ResNet

[\frac{\partial L}{\partial x}

0.8 × 0.0002
]

[
=0.00016
]

Almost zero ❌

Network stops learning.

With ResNet

[\frac{\partial L}{\partial x}

0.8(0.0002 + 1)
]

[
=0.8 × 1.0002
]

[
=0.80016
]

Gradient preserved ✓

STEP-4: FINAL ANSWER

👉 **FINAL ANSWER**

ResNet introduces skip connection:

[
\boxed{y = F(x) + x}
]

This allows gradient:

$$
\boxed{\frac{dy}{dx} = \frac{dF(x)}{dx} + 1}
$$

So gradient never becomes zero, solving degradation and vanishing gradient problems and enabling very deep networks (50, 101, 152 layers).

⭐ LONG QUESTION — 9

🔶 Compare Autoencoder, Sparse Autoencoder and CNN. Explain working principle, objective function and applications.

STEP-1: THEORY EXPLANATION (FROM ZERO → EXAM LEVEL)

First understand the core idea

All three are neural networks — but **their goals are different**

Model	Main Goal
Autoencoder	Compress data
Sparse Autoencoder	Learn meaningful features
CNN	Recognize patterns in images

---

1) Autoencoder (AE)

Intuition

Like compressing a ZIP file and decompressing it back.

```
Image → Compress → Hidden code → Reconstruct image
```

Network tries:

[
Output ≈ Input
]

So it learns internal representation.

Structure Diagram

```
Input x
   ↓
[ Encoder ]
   ↓
Latent vector h
   ↓
[ Decoder ]
   ↓
Reconstructed x̂
```

What it learns

General patterns of data

Examples:

- noise removal
- compression
- dimensionality reduction

---

2) Sparse Autoencoder (SAE)

Problem in normal AE

Network may just copy input directly → no real learning.

Solution

Force only few neurons to activate.

Human brain analogy:

```
Normal AE : many neurons active
Sparse AE : few neurons active (efficient brain)
```

Sparse Diagram

```
Hidden layer activations

Normal : [● ● ● ● ● ●]
Sparse : [● ○ ○ ○ ● ○]
```

Result

Learns meaningful features:

- edges
- strokes
- object parts

---

3) Convolutional Neural Network (CNN)

Intuition

Instead of compressing data, CNN detects patterns.

Human vision:

```
see edges → shapes → object
```

CNN does same.

CNN Pipeline

```
Image → Convolution → Pooling → Fully Connected → Class
```

What it learns

Layer	Learns
early	edges
middle	textures
deep	object

STEP-2: MATHEMATICAL FORMULAS & SYMBOLS

Autoencoder Loss

$$
L_{AE} = \frac{1}{m}\sum ||x - \hat{x}||^2
$$

Symbol	Meaning
x	input
$\hat{x}$	reconstructed output
m	number of samples

Sparse Autoencoder Loss

$$
L = L_{AE} + \beta \sum KL(\rho || \hat{\rho})
$$

KL divergence:

$$
KL(\rho||\hat{\rho})
$$

$$
\rho \log \frac{\rho}{\hat{\rho}}
+
(1-\rho)\log\frac{1-\rho}{1-\hat{\rho}}
$$

Symbol	Meaning
ρ	desired sparsity
$\hat{\rho}$	average activation
β	sparsity weight

CNN Convolution

[
Feature = I * K
]

[
Feature(i,j)=\sum I(i+m,j+n)K(m,n)
]

Symbol	Meaning
(I)	image
(K)	kernel

STEP-3: PRACTICAL COMPARISON (NUMERICAL INTUITION)

Suppose we have handwritten digit dataset.

Autoencoder

Learns compressed vector:

```
784 pixels → 32 numbers
```

Reconstruction good but not classification.

Sparse Autoencoder

Learns:

- strokes
- curves
- digit parts

Better features for classifier.

CNN

Directly recognizes:

```
Image → Digit = 5
```

Best for recognition.

STEP-4: FINAL ANSWER

👉 **FINAL ANSWER**

Model	Purpose	Key Idea	Loss
Autoencoder	Compression	reconstruct input	MSE
Sparse AE	Feature learning	sparse neurons	MSE + KL
CNN	Pattern recognition	convolution filters	classification loss

$$
\boxed{\text{AE = compression, SAE = feature learning, CNN = recognition}}
$$

⭐ LONG QUESTION — 10

🔶 Explain the complete Deep Learning training pipeline: Forward pass → Loss → Backpropagation → Optimizer → Weight update.

STEP-1: THEORY EXPLANATION (FROM ZERO → EXAM LEVEL)

Big Picture

Training a neural network is like teaching a student.

Real Life	Neural Network
Student answers	Prediction
Teacher checks	Loss function
Teacher explains mistake	Backpropagation
Student improves	Weight update

So learning is a **repeating cycle**.

Full Learning Cycle Diagram

```
          (Training Data)
                ↓
           Forward Pass
                ↓
         Predicted Output
                ↓
            Loss Function
                ↓
         Backpropagation
                ↓
             Optimizer
                ↓
          Update Weights
                ↓
             Repeat
```

This loop continues until error becomes minimum.

STEP 1 — Forward Pass (Prediction)

Network computes output from input.

Example CNN:

```
Image → Conv → ReLU → Pool → FC → Output
```

Each neuron performs:

[
z = Wx + b
]

$$
a = \sigma(z)
$$

We get prediction:

$$
\hat{y}
$$

STEP 2 — Loss Function (Measure Error)

We compare prediction with actual answer.

If prediction wrong → high loss
If correct → low loss

Example:

$$
L = \frac{1}{2}(y-\hat{y})^2
$$

This tells **how bad the model is**

STEP 3 — Backpropagation (Find Mistake Source)

Now we ask:

> Which weight caused this error?

We send error backward layer by layer.

```
Output layer → Hidden layer → Input layer
```

This calculates gradient (error contribution).

STEP 4 — Optimizer (Decide Correction Size)

Optimizer decides:

How big step should we take to fix weights?

Different optimizers:

- SGD → small steps
- Momentum → smooth steps
- Adam → adaptive steps

STEP 5 — Weight Update (Learning)

Weights updated:

$$
New\ weight = Old\ weight - Learning\ rate \times Gradient
$$

Network becomes slightly smarter.

Training Loop

```
Epoch 1 → wrong
Epoch 2 → better
Epoch 3 → good
Epoch 100 → learned
```

Loss graph:

```
Loss
 |
 |\
 | \
 |  \
 |   \____
 +--------- Epochs
```

STEP-2: MATHEMATICAL FORMULAS WITH SYMBOL NAMES

Forward Pass

$$
z = Wx + b
$$

[
a = \sigma(z)
]

Symbol	Meaning
(W)	weight matrix
(x)	input
(b)	bias
(a)	activation

Loss Function

[
L = \frac{1}{2}(y-\hat{y})^2
]

Symbol	Meaning
(L)	loss
(y)	true output
(\hat{y})	predicted output

Backpropagation Gradient

[
\frac{\partial L}{\partial W}

\frac{\partial L}{\partial a}
\frac{\partial a}{\partial z}
\frac{\partial z}{\partial W}
]

Symbol	Meaning
(\partial)	partial derivative
gradient	error contribution

Optimizer Update Rule

$$
W_{new} = W_{old} - \eta \nabla L
$$

Symbol	Meaning
(\eta) (eta)	learning rate
(\nabla L)	gradient

STEP-3: NUMERICAL DEMONSTRATION

Given:

$$
x=1,\quad W=0.5,\quad b=0,\quad y=1,\quad \eta=0.1
$$

Forward Pass

$$
z = 0.5(1)=0.5
$$

$$
\hat{y}=\sigma(0.5)=0.622
$$

Loss

$$
L=\frac{1}{2}(1-0.622)^2
$$

$$
=0.5(0.378)^2
$$

$$
=0.5(0.1428)
$$

$$
L=0.0714
$$

Gradient

$$
\frac{\partial L}{\partial W}=(\hat{y}-y)\hat{y}(1-\hat{y})x
$$

$$
=(0.622-1)(0.622\times0.378)(1)
$$

$$
=(-0.378)(0.235)
$$

$$
=-0.0888
$$

Weight Update

$$
W_{new}=0.5-0.1(-0.0888)
$$

$$
W_{new}=0.5089
$$

Network improved ✓

STEP-4: FINAL ANSWER

👉 **FINAL ANSWER**

The deep learning training pipeline follows:

$$
\boxed{
Forward\ Pass \rightarrow Loss\ Calculation \rightarrow Backpropagation \rightarrow Optimizer \rightarrow Weight\ Update \rightarrow Repeat
```
```

}
]

This iterative process minimizes error and enables the network to learn patterns from data.

You now have **complete preparation material**:

- 20 important 5-mark questions ✓
- 10 important 10-mark questions ✓

This covers almost the entire ADL paper pattern.
